

MERN Stack Coding Assessment

Stack: MongoDB · Express · React · Node.js

Problem Statement: "DeskFlow" — A Support Ticket Triage Board

Build a support ticket system. Customers submit tickets with a priority. Support agents see a board grouped by status, can move tickets between statuses, and each ticket tracks how long it has been open. Tickets that exceed their response time target are flagged as breaching SLA.

Objective

Test your ability to design a small MERN application involving derived fields, time-based logic, status transitions with rules, and an interactive board-style UI.

Part 1 — Backend

Data Model: Ticket

Field	Type	Notes
subject	String	Required
description	String	Required
customerEmail	String	Required, valid email format
priority	String	One of: low , medium , high , urgent
status	String	One of: open , in_progress , resolved , closed . Defaults to open .
createdAt	Date	Auto-generated
resolvedAt	Date	Set automatically when status moves to resolved ; cleared if moved back.

Server-side Rules

- Status transitions allowed: open → in_progress → resolved → closed . Skipping forward (e.g. open → resolved) is not allowed. Moving backwards is allowed only one step (e.g. resolved → in_progress), and must clear resolvedAt .
- Each ticket has a response time target based on priority:
 - urgent — 1 hour
 - high — 4 hours
 - medium — 24 hours
 - low — 72 hours
- Compute two derived fields on every ticket response:
 - ageMinutes — minutes between createdAt and now (or resolvedAt if resolved).
 - slaBreached — boolean. True if a ticket is still unresolved past its target, or if it was resolved after its target.

API Endpoints

Method	Endpoint	Description
POST	<code>/tickets</code>	Create a ticket.
GET	<code>/tickets</code>	List tickets. Supports <code>?status</code> , <code>?priority</code> , and <code>?breached=true</code> filters (combinable).
PATCH	<code>/tickets/:id</code>	Update a ticket. Used to change status. Must enforce the transition rules above.
DELETE	<code>/tickets/:id</code>	Delete a ticket.
GET	<code>/tickets/stats</code>	Returns aggregate counts per status, per priority, and the number of SLA-breached tickets currently open.

Sample Response — single ticket

```
{
  "_id": "65f...",
  "subject": "Cannot log in",
  "priority": "high",
  "status": "in_progress",
  "createdAt": "2026-05-20T08:00:00.000Z",
  "ageMinutes": 312,
  "slaBreached": true
}
```

Notes

- Validate input and return appropriate error responses for invalid data.
- Invalid transitions (e.g. `open → closed`) must be rejected with a clear error.
- The derived fields must be computed on the server at read time.

Part 2 — Frontend

Layout

- A single-page UI with a **board view** at the top and a **create-ticket form** elsewhere on the page (modal or side panel — your choice).

Board View

- Four columns, one per status: **Open**, **In Progress**, **Resolved**, **Closed**.
- Each ticket card shows subject, priority (visually distinguishable — colored badge or similar), age (e.g. "3h 12m"), and an indicator if SLA is breached.
- Each card has buttons or controls to move it to an adjacent status. Disallowed transitions should not be available as actions in the UI.
- Moving a ticket must update the board without a page reload.

Filters

- Controls above the board to filter by priority and to show only SLA-breached tickets.

Stats Strip

- A small strip somewhere on the page showing total tickets per status and the count of currently breached tickets, fetched from the stats endpoint.

Create Ticket

- A form to submit a new ticket (subject, description, customer email, priority). On success, the new ticket appears in the appropriate column without a reload.

General

- Handle loading and error states.

- Show inline form errors, not generic alerts.

Styling is up to you — clean and functional is enough. No UI library required.

Deliverables

- A public **GitHub repository** with both `backend/` and `frontend/` folders.
- The project must be **deployed and publicly accessible**:
 - **Frontend**: Vercel, Netlify, or Render Static Site
 - **Backend**: Render, Railway, or Fly.io (use MongoDB Atlas free tier)
- Submit two links: the **GitHub repo URL** and the **live deployed frontend URL**.
- A `README.md` is *optional*.

Bonus (Optional)

Make the ticket cards **drag-and-droppable** between columns. Dropping a card on a column should attempt the corresponding status change and respect the transition rules — invalid drops should snap back with a brief error indicator.

Evaluation Notes

Reviewers will test your submission against these criteria:

- Your API must respond under **3 seconds** on the deployed environment.
- Invalid input (missing fields, bad email, unknown priority, unknown status) must return a `400` with a clear error message — not a generic crash.
- Attempting an invalid status transition (e.g. `open → resolved` directly) must be rejected with a clear error.
- Moving a ticket to `resolved` must set `resolvedAt` automatically. Moving it back from `resolved` must clear `resolvedAt`.
- The `slaBreached` flag must correctly reflect the priority-based target — a high-priority ticket created 5 hours ago and still unresolved must show `slaBreached: true`.
- For resolved tickets, `ageMinutes` must stop growing — it should reflect the time between creation and resolution, not the time since creation.
- The combined filters (priority + breached) must work together, not just individually.
- The UI must not offer disallowed transitions — a card in `open` must not have a button to move it directly to `resolved`.
- Moving a ticket between columns must update the board without a full page reload.
- The deployed frontend must successfully talk to the deployed backend — no `localhost` URLs, no CORS errors in the console.

Plagiarism check: All submitted GitHub repositories will be run through automated plagiarism and code-similarity detection. Submissions copied from public repos, other candidates, or AI-generated boilerplate without meaningful modification will be **disqualified**. You may reference documentation, but the code must be your own.

Submission Notes

- You may use any npm packages you find helpful.
- Use MongoDB Atlas free tier for the database.
- Commit frequently. Partial working solutions score better than incomplete "perfect" attempts.

Good luck!